

Code Explanation

Code Explanation: Comprehensive Test Suite for Customer Order Analytics Glue Job

Overview

This is a **pytest-based test suite** for a PySpark AWS Glue job that processes customer and order data. It validates all functional requirements (FR) and technical requirements (TR) using mocked AWS services.---

Step-by-Step Breakdown

****1. Imports and Setup (Lines 1-40)****

- **Purpose**: Import testing frameworks, AWS mocking tools, PySpark libraries, and the application code under test- **Key Libraries**: - `pytest`: Test framework - `moto`: AWS service mocking (S3) - `pyspark.sql`: DataFrame operations - Application modules: `SparkContextManager`, `S3Validator`, `DataReader`, `DataWriter`, `DataCleaner`, `SCDType2Handler`, `CustomerOrderAnalytics`---

****2. Test Fixtures (Lines 42-130)****

****2.1 Spark Session Fixture**** (Lines 42-52)- Creates a local Spark session for testing- Configured with 2 partitions for performance- Session-scoped: created once, reused across all tests- Automatically stopped after all tests complete##### ****2.2 Component Fixtures**** (Lines 54-72)- Instantiate application classes for testing: - `s3_validator`: Validates S3 paths - `data_reader`: Reads data from S3 - `data_writer`: Writes data to S3 - `data_cleaner`: Cleans data (removes nulls, duplicates) - `scd2_handler`: Implements Slowly Changing Dimension Type 2 logic##### ****2.3 Sample Data Fixtures**** (Lines 74-130)- `sample_customer_data`: Clean customer records (5 customers)- `sample_order_data`: Order records with pricing and quantities (5 orders)- `sample_customer_data_with_nulls`: Dirty data containing: - NULL values - String literals "Null" and "null" - Duplicate records---

****3. S3 Validator Tests (Lines 132-188)****

****3.1 Valid Path Test**** (Lines 132-143)- **Requirement**: Validate correct S3 path format- **Method**: Uses `@mock_s3` decorator to simulate S3- **Verification**: Returns `True` for valid path `s3://adif-sdlc/test/`##### ****3.2 Invalid Path Test**** (Lines 145-150)- **Verification**: Returns `False` for malformed path `invalid-path`##### ****3.3 Path Exists Test**** (Lines 152-168)- **Method**: Creates mock bucket and object- **Verification**: Returns `True` when objects exist at path##### ****3.4 Path Not Exists Test**** (Lines 170-182)- **Verification**: Returns `False` for empty S3 prefix---

****4. Data Cleaning Tests (Lines 190-272)****

Maps to FR-CLEAN-001 and TR-INGEST-002##### ****4.1 Remove NULL Values**** (Lines 190-210)- **Test**: Verifies NULL values are removed from all columns- **Assertion**: `null_count == 0` after cleaning##### ****4.2 Remove 'Null' Strings**** (Lines 212-227)- **Test**: Removes string literals "Null" and "null"- **Assertion**: No rows contain these string values##### ****4.3 Remove Duplicates**** (Lines 229-242)- **Test**: Ensures duplicate records are eliminated- **Assertion**: Row count equals distinct count##### ****4.4 Complete Cleaning Pipeline**** (Lines 244-260)- **Test**: Validates all cleaning rules applied together- **Expected Result**: Only 1 clean record (C001) remains from 7 input records---

****5. SCD Type 2 Tests (Lines 262-314)****

****Maps to FR-SCD2-001**##### **5.1 Add SCD Columns** (Lines 262-283)- ****Test****: Verifies 4 new columns added: - `isactive` (Boolean) - `startdate` (Timestamp) - `enddate` (Timestamp) - `opts` (Timestamp - operation timestamp)##### **5.2 Initial Load Values** (Lines 285-302)- ****Test****: Validates initial load behavior- ****Expected Values****: - `isactive = True` - `startdate` is set - `enddate = None` - `opts` is set---**

****6. Schema Validation Tests (Lines 304-368)****

****6.1 Customer Schema Test** (Lines 304-328)**Maps to FR-INGEST-001** - ****Validates****: 4 fields (custid, name, emailid, region)- ****Data Types****: All StringType##### **6.2 Order Schema Test** (Lines 330-358)**Maps to FR-INGEST-002** - ****Validates****: 5 fields (orderid, itemname, priceperunit, qty, date)- ****Data Types****: String, Decimal(10,2), Integer, String---**

****7. Business Logic Tests (Lines 360-442)****

****7.1 Customer Aggregate Spend** (Lines 360-398)**Maps to FR-AGG-001** - ****Logic****: `sum(PricePerUnit \* Qty)` grouped by CustId, Name- ****Verification****: - C001: $999.99 + (29.99 \times 2) = 1059.97$ - C002: 79.99 - C003: $299.99 \times 2 = 599.98$ ##### **7.2 Order Summary Generation** (Lines 400-432)**Maps to FR-SUMMARY-001** - ****Output Columns****: orderid, custid, name, itemname, totalamount, date- ****Verification****: - O001: $999.99 \times 1 = 999.99$ - O002: $29.99 \times 2 = 59.98$ ---**

****8. Configuration Tests (Lines 434-476)****

****8.1 S3 Path Validation** (Lines 434-452)- ****Verifies TRD-specified paths****: - Input: `s3://adif-sdlc/sdlc\_wizard/customerdata/` - Output: `s3://adif-sdlc/analytics/customeraggregatespend/` - Curated: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`##### **8.2 Column Normalization** (Lines 454-476)- ****Test****: Converts all column names to lowercase- ****Method****: `df.toDF(\*[c.lower() for c in df.columns])`---**

****9. Hudi Configuration Test (Lines 478-506)****

****Maps to FR-SCD2-001** - ****Validates Hudi parameters****: - Record Key: `custid` - Precombine Field: `opts` - SCD columns present for upsert operation---**

****10. Data Quality Tests (Lines 508-542)****

****10.1 No NULLs After Cleaning** (Lines 508-518)- ****Iterates all columns**** to verify zero NULL values##### **10.2 Positive Amounts** (Lines 520-530)- ****Validates****: `priceperunit > 0` and `qty > 0`---**

****11. Edge Case Tests (Lines 532-580)****

****11.1 Spark Context Initialization** (Lines 532-546)- ****Mocks****: SparkContext and GlueContext- ****Verifies****: Proper initialization without errors##### **11.2 Customers with No Orders** (Lines 548-564)- ****Test****: Empty order DataFrame- ****Expected****: Returns empty aggregate result (no crash)##### **11.3 Join Integrity** (Lines 566-580)- ****Test****: Verifies all orders match customer records- ****Assertion****: No NULL customer names in summary---**

****12. Test Execution** (Lines 582-583)**

- **Entry Point**: Runs pytest with verbose output and short traceback- **Command**:
`pytest.main(__file__, "-v", "--tb=short")`---

Key Testing Patterns

- 1. **Mocking**: Uses `@mock_s3` for AWS service simulation
- 2. **Fixtures**: Reusable test data and component instances
- 3. **Traceability**: Each test maps to specific FR/TR requirements
- 4. **Isolation**: Each test is independent and can run in any order
- 5. **Assertions**: Validates data counts, schema types, and calculated values
- 6. **Edge Cases**: Tests empty datasets, NULL handling, and join failures---

Requirements Coverage

Requirement Test Coverage ----- -----	FR-INGEST-001 Customer schema validation
FR-INGEST-002 Order schema validation	FR-CLEAN-001 NULL/duplicate removal tests
FR-SCD2-001 SCD Type 2 column tests	FR-AGG-001 Aggregate spend calculation
FR-SUMMARY-001 Order summary generation	